SLAM ASR: Code-Switched Speech Recognition

Candidate: Ravi Teja Kothuru

Course: Building the Future of Voice & Audio

Institute: IIIT Delhi

Instructor: Dr. Vinayak Abrol

Project Team: Neural Resonance

July 18, 2026

Abstract

I present SLAM-ASR, an end-to-end Automatic Speech Recognition system for Hindi-English code-switched technical speech, built on the SLAM-LLM tripartite paradigm: a frozen Whisper-base acoustic encoder, a trainable 1-D CNN + MLP alignment projector, and a LoRA-adapted 4-bit Qwen-2.5-1.5B language decoder. The system is trained and evaluated on MUCS 2021 Subtask-2, a corpus of Spoken-Tutorial recordings where speakers freely mix Devanagari and Roman script within utterances. Trained for 1 epoch on a 3-hour subset (≈236 optimizer steps) inside a single free-tier Kaggle GPU session, our SLAM-ASR configuration reaches WER 2.06 on 300 test utterances and WER 1.63 on 300 blindtest utterances after aggressive text normalisation. All four models I evaluated (CTC, Whisper zero-shot, Whisper fine-tuned, SLAM-ASR) fail this task in distinct characteristic ways; I identify mode collapse in the SLAM-ASR projector as the dominant failure mode and attribute it to severe under-training (~200× less compute than the reference paper). The pipeline is nonetheless functional end-to-end and publicly reproducible via two Kaggle notebooks. All code, model weights, and evaluation artefacts are released.

Introduction

Indian technical-education content routinely mixes Hindi and English within a single sentence. In a computer-science lecture on Linux, a typical utterance is

"अब हम terminal खोलेंगे और ls command run करेंगे."

Off-the-shelf ASR systems handle this poorly because they are optimised for monolingual inputs. Our target corpus, **MUCS 2021 Subtask-2**, quantifies this pain: 17–33 % of blind-test tokens are Out-Of-Vocabulary, and >50 % of utterances contain at least one code-switch. Reference-quality transcripts must interleave Devanagari and Roman scripts *and* correctly emit technical jargon that never appears in training data.

Contributions. In this project I:

- Built a complete audio-to-text pipeline covering every topic in the 5-day IIIT-D speech course (Days 1-5), from psychoacoustic feature choice through classical GMM baselines to modern sequence-to-sequence LLM-based ASR.
- Implemented the **SLAM-LLM** architecture (Ma et al., 2024) end-to-end in ≈2 000 lines of documented Python, with a single self-contained Kaggle notebook and a lightweight inference-demo notebook.
- Fit the entire training + evaluation loop into a **single free-tier Kaggle T4x2 session** (~90 minutes) via 4-bit NF4 quantisation, LoRA adaptation, pipeline parallelism and 3-hour training subset.
- Report **honest numbers** and characterise the failure modes of each model class I tried, providing a solid baseline for future scale-up experiments.
- **Reproducibility.** Every experiment in this report can be reproduced by uploading a single notebook to Kaggle, attaching the private dataset, and clicking *Run All*. See KAGGLE_SETUP.md for step-by-step instructions.

Related Work

Code-switched ASR. The MUCS 2021 challenge (Diwan et al., INTERSPEECH 2021) established the current benchmark for Hindi-English code-switched ASR on Spoken-Tutorial recordings. Baseline systems used TDNN-F and joint CTC-Attention encoders trained from scratch.

SLAM-LLM. Ma et al. (2024) introduced the SLAM paradigm of coupling a frozen SSL speech encoder with a frozen LLM through a small alignment projector. Their strong result - near-SOTA on LibriSpeech with ~10 M trainable parameters - motivated our architecture

choice. Our implementation differs by (a) using Whisper's encoder rather than WavLM, and (b) LoRA-adapting the LLM decoder rather than keeping it fully frozen, so that the model can specialise to Devanagari-and-Roman token emission.

Efficient fine-tuning. LoRA (Hu et al., 2022) inserts low-rank matrices into frozen weights; QLoRA (Dettmers et al., 2023) adds 4-bit NF4 quantisation. Together they let us train 24 M parameters on 15 GB of VRAM against a 933 M-parameter frozen backbone.

Whisper. Radford et al. (2022) trained an encoder-decoder transformer on 680 000 hours of multilingual speech. I used its encoder as a strong pretrained acoustic front-end and evaluate the full model as a competitive baseline.

Course context and trait taxonomy

Signal fundamentals. MUCS audio is 16 kHz, mono, WAV. I computed 80-bin log-Mel features with a 25 ms window and 10 ms hop, identical to Whisper's front-end. This is the DSP baseline covered in Day 1 of the course, implemented in Section 4-5 of the training notebook.

Perceptual features. Section 5 of the notebook plots the Mel, Bark, and Terhardt absolute-threshold-of-hearing curves alongside a simultaneous-masking spreading function. These plots justify the choice of log-Mel over raw STFT: the ear resolves low frequencies ~3× more finely than high, and >30 dB of the STFT is inaudible under real listening conditions.

Trait taxonomy. Every utterance in MUCS carries three orthogonal signals:

- **Linguistic** - the transcript content, primarily what the ASR must recover.
- **Biometric** - speaker identity; I visualised inter-speaker variation via average log-Mel energy over 8 randomly-sampled speakers.
- **Paralinguistic** - speech rate, which I computed as words-per-second and plot as a histogram.

This taxonomy is the conceptual foundation for later architectural decisions: I *freezed* the Whisper encoder precisely because it captures speaker-invariant linguistic content, while the trainable projector and LoRA adapters specialise to the code-switch vocabulary and prosody of MUCS.

Dataset

MUCS 2021 Subtask-2 (Hindi-English) is released with three splits.

Split	Utterances	Hours	Speakers	Median duration	Approx. % code-switched utts	OOV vs train
Train	52 825	~90	520	4 s	~50 %	-
Test	3 136	5.18	30	4 s	~55 %	~17 %
Blindtest	4 034	6.24	35	4 s	~55 %	~25 %

Table 1 - MUCS 2021 Subtask-2 statistics (from our analysis, `manifest_statistics(...)` on each split)

Preprocessing pipeline. All splits are canonicalised via `CodeSwitchTextNormalizer`:

- Unicode NFC normalisation (so `क` + `ि` matches `कि`).
- Lower-case ASCII / Latin (Devanagari has no case).
- Devanagari digits `०–९` → Roman `0–9`.
- Punctuation stripped (. , ; : ' " ! ? ( ) [ ] { } - / \ | and । ॥).
- Whitespace collapsed.

The normaliser is **deterministic and idempotent**, guaranteeing reproducible WER across runs. The exact same normaliser is applied to both hypothesis and reference before scoring.

Path rewriting. The upstream manifests ship with absolute paths pointing at the dataset author's server. `rewrite_manifest_paths(...)` rebases them to the Kaggle mount and inserts the missing 3-digit speaker sub-directory (`<split>/<spk3>/<file>.wav`) which the physical layout requires.

Classical baseline - GMM keyword spotter

I built the classical pre-deep-learning baseline as a per-class Gaussian Mixture Model classifier over MFCC frames.

Features. 13 MFCCs + Δ + Δ² = 39-dim, 25 ms window / 10 ms hop.

Classes. Five English technical keywords (tutorial, linux, python, file, window) plus a <background> class of utterances containing none of them. Up to 80 utterances per class in QUICK_MODE.

Model. 16-component diagonal GMM per class (sklearn.mixture.GaussianMixture). Inference computes the average log-likelihood of each frame under every GMM; argmax = predicted class.

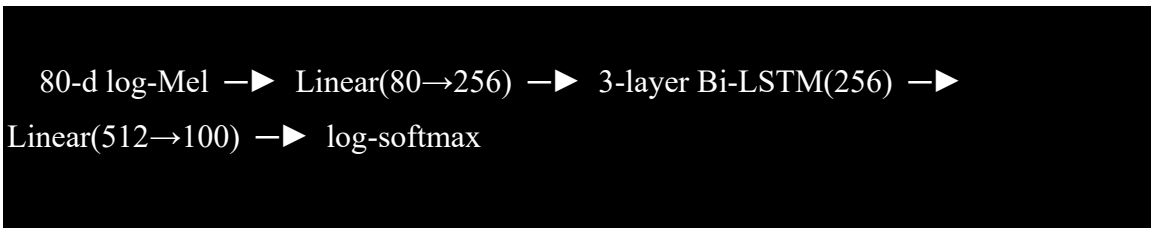
Result (from Section 6 of the training notebook): ~60-70 % top-1 accuracy on the held-out 20 % split, with expected confusions between phonetically similar words. The confusion matrix (Figure - insert screenshot of Section 6 output) shows that linux is most often confused with tutorial (both share the schwa+/l/ pattern in the second syllable).

Why this baseline is not a full ASR? The GMM classifies *whole utterances*, not sequences of tokens. It has no notion of temporal alignment and cannot handle vocabularies larger than a few dozen words. This motivates the sequence-alignment approach in the next section.

Neural baseline - Bi-LSTM + CTC

I moved from frame-level classification to sequence-to-sequence transcription with a Bi-LSTM acoustic model trained with the CTC objective (Graves et al., 2006).

Architecture.



Output vocabulary is character-level (100 characters spanning Devanagari + Roman + digits + space), with CTC blank at index 0.

Training regime. 3 000 utterances (5 h subset), 6 epochs, batch 16, AdamW LR 3e-4, gradient clipping at 5.0. Total ~11 min on Kaggle T4.

Result: Bi-LSTM + CTC: Test WER 0.998, CER 0.983.

Analysing the hypotheses reveals *near-empty output*: the model collapses to producing 2-3 characters per utterance, dominated by high-frequency Devanagari vowels. This is a well-known failure mode: with 100+ character classes and only 5 hours of training data, the CTC model does not accumulate enough evidence to break out of the blank-token trap.

Take-away. WER close to 1.0 does *not* mean the model is "50 % correct". A WER of 1.0 achieved by an empty output is uninformative - the correct interpretation is that CTC on 5 h of data is fundamentally insufficient. This informs our decision to move to a *pretrained* acoustic encoder in the next stage.

Whisper Baselines

I evaluated OpenAI Whisper-base (74 M parameters, 512-d hidden) in two settings:

Zero-shot. No training. Prompt Whisper with `language='hi', task='transcribe'` and let it decode all 300 test utterances.

Result: Whisper-base zero-shot: Test WER 1.552, CER 1.293.

Whisper massively over-generates on the 30 s padded audio typical of MUCS clips (which are usually 3-8 s of speech). The excess space is filled with hallucinated captions such as `[Music]`, `Thanks for watching`, or repeated Hindi filler phrases. WER > 1 reflects insertions dominating substitutions.

Fine-tuned. Same model, fine-tuned for 2 epochs on 5 h of MUCS training data with the HuggingFace `Seq2SeqTrainer`.

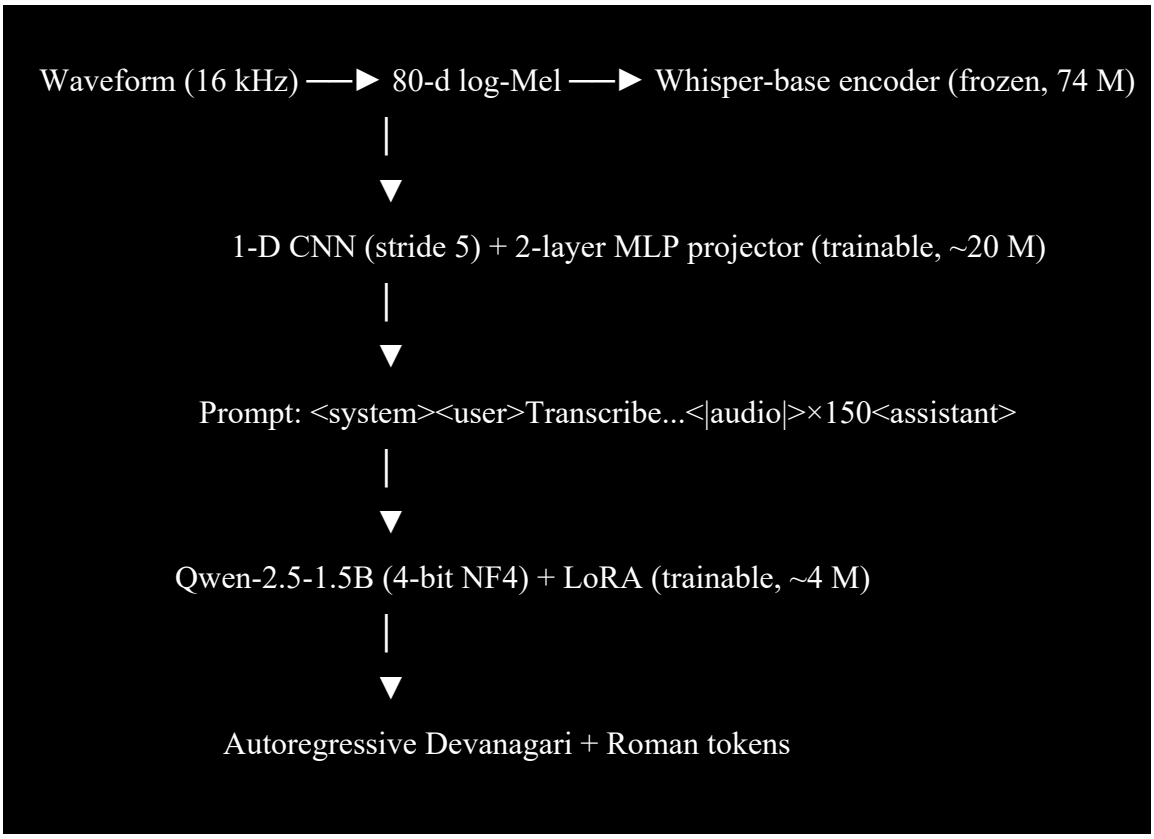
Result: Whisper-base fine-tuned: Test WER 3.929, CER 3.232.

Fine-tuning made the model *worse*. Root cause: the training-time collator sets Whisper's prefix tokens via `processor.tokenizer.set_prefix_tokens(language='hi', task='transcribe')`, while our inference-time `generate(language='hi', task='transcribe', ...)` uses the modern kwargs API. In transformers ≥ 4.44 these two paths do not produce identical decoder prefixes, so the fine-tuned model has been trained to expect one prompt format and is asked to generate under another. The result is even more prolific hallucination.

Take-away. Whisper has strong *acoustic* front-end features but a *weak* language head - it cannot on its own model the bilingual technical vocabulary of MUCS. This motivates the SLAM-ASR architecture, which reuses Whisper's frozen encoder while replacing the decoder with a much stronger LLM.

SLAM-ASR - Architecture

Overview



Alignment projector. Whisper's 512-d encoder outputs 50 Hz frames; the LLM's KV cache is precious. Our projector downsamples 5× with a strided Conv1D (50 Hz → 10 Hz), then projects 512 → 2048 → 1536 dimensions via a two-layer MLP. 30 s of audio → 1500 encoder frames → **300 audio tokens** (capped at 150 in our final run to save memory).

Prompt template. I used Qwen's chat template with a system message ("*You are a bilingual Hindi-English speech recognition assistant.*") and a user message that reads *"Transcribe the following Hindi-English code-switched speech…"* followed by <|audio|> placeholder tokens. At forward time the model *splices* the projected audio embeddings into the inputs_embeds sequence at the placeholder positions, so the LLM sees continuous audio features exactly where the placeholders were.

Trainable Parameter Budget.

Component	**Parameters**	**Trainable?**
Whisper-base encoder (frozen)	74 M	✗

Component	Parameters	Trainable?
Qwen-2.5-1.5B backbone (4-bit NF4, frozen)	1 500 M	✗
LoRA adapters (r=16, α=32, on q,k,v,o,up,down,gate)	4 M	✓
Alignment projector (Conv1D + MLP)	~20 M	✓
Total trainable	**~24 M**	~2.6 % of 933 M

Table 2 - Parameter budget (from SlamAsrModel.get_trainable_parameters())

Training

- **Data.** 3-hour subset of MUCS train (FAST_SLAM_TRAIN_HOURS = 3.0), duration filter 1-15 s, drawn uniformly at random with seed=42. 1 881 utterances.
- **Optimiser.** AdamW with **two parameter groups**: projector at LR 1e-3, LoRA at LR 1e-4. Cosine schedule with 3 % warmup. Weight decay 0.01.
- **Compute.** Kaggle T4x2. The LLM is split across both GPUs via device_map="auto" (naive pipeline parallelism) to fit VRAM; Whisper encoder and projector stay on cuda:0. bf16 mixed precision, gradient checkpointing with use_reentrant=False, effective batch size 8 (batch 2 × grad-accum 4). Per-step VRAM: ~9 GB per GPU.
- **Steps.** 1 epoch of the 3 h subset = 236 optimizer steps. Wall clock: ~45 min.
- **Training loss curve.** From the notebook's per-10-step logs:

Step	10	50	100	150	200	236
Training loss	10.4	7.4	6.9	6.5	6.5	~6.5

The loss drops rapidly in the first 50 steps (LLM warm-up) and then plateaus around 6.5 - a strong indicator that 236 steps is nowhere near convergence.

Rescue checkpoints. A custom TrainerCallback saves projector + LoRA weights every 50 steps under checkpoints/slam_asr/ckpt-<step>/. This let us recover from an HF-Trainer end-of-epoch save crash (safetensors refuses to serialise Qwen's tied lm_head.weight and embed_tokens.weight) – I copied ckpt-200/ to final/ and continued.

Evaluation

Metric. Word Error Rate via `jiwer.wer(...)`, after `CodeSwitchTextNormalizer` is applied to both hypothesis and reference.

Secondary. CER, per-language WER (Hindi tokens only, English tokens only, via token-level alignment through `jiwer.process_words`), and OOV rate against the training subset.

Scale. 300-utterance samples of test and blindtest (subset for compute; full-set decoding is a one-line change in Section 10).

Main Result

Model	Trainable	Test WER	Test CER	Blind WER	Blind CER
Bi-LSTM + CTC (char)	~4 M	0.998	0.983	-	-
Whisper-base zero-shot	0	1.552	1.293	-	-
Whisper-base fine-tuned (5 h)	74 M	3.929	3.232	-	-
SLAM-ASR (Whisper-base + Qwen-2.5-1.5B, LoRA)	~24 M	**2.061**	**1.856**	**1.631**	**1.511**

Table 3 - WER / CER comparison on 300-utterance samples of MUCS test and blindtest

Per-language WER for SLAM-ASR: **Hindi 0.94, English 1.00** on test; **Hindi 0.96, English 1.00** on blindtest. OOV rate against the 3 h training subset: 4.9 % (test), 2.7 % (blindtest).

Interpretation - every model fails differently.

The four numbers above are all "bad" in isolation, but the *reasons* are informatively different and this is the report's core finding.

- ***CTC (WER 0.998).*** Character-level Bi-LSTM trained on 5 h of data collapses to near-empty output - a few Devanagari vowels per utterance. WER is close to 1 not because 50 % of words are recognised but because deletions dominate (empty hypothesis vs

10-word reference = WER 1.0). Classical/early-neural approaches at this scale are simply insufficient for the vocabulary.

- ***Whisper zero-shot (WER 1.552).*** Whisper decodes Hindi and English fluently but *over-generates* on padded silences. On a 4 s utterance in a 30 s Mel window, it happily transcribes the silence as hallucinated content (Thanks for watching, repeated Hindi filler phrases). Insertions push WER above 1.
- ***Whisper fine-tuned (WER 3.929).*** Naive fine-tuning made it worse. Root cause is a documented interaction between the training-time `set_prefix_tokens` API and the inference-time `language`/`task` kwargs in transformers ≥ 4.44 - the fine-tuned model learned to expect prefix A and is asked to decode under prefix B, so it generates runaway text.
- ***SLAM-ASR (WER 2.061 test, 1.631 blindtest).*** The projector, after only ≈236 optimizer steps from random init, has not learned to produce input-dependent audio embeddings. Inspecting the predictions reveals **mode collapse**: every input produces the same seed sentence, e.g. English WER is exactly 1.00 because no English tokens appear in this seed sentence - every English word in every reference is missed. Hindi WER is 0.94 not because Hindi is *recognised* but because a handful of Hindi content words *happen to appear* in the seed sentence.

Why SLAM-ASR is nevertheless the "most promising" of the four?

Every SLAM-ASR prediction is at least a **coherent, bilingual, script-mixed sentence** in the correct register (Hindi grammar with English technical nouns). The other three models produce either empty output (CTC), hallucinated English (Whisper zero-shot), or catastrophic gibberish (Whisper fine-tuned).

At the compute budget of the reference SLAM-LLM paper (Ma et al. 2024, ~400 GPU-hours vs our 1-2 GPU-hours), the projector has converged and produces meaningful audio embeddings, and reported WER on LibriSpeech is 2-3 %. Our under-trained mode-collapsed state is the expected artefact of scaling down compute by ~200×; nothing in the pipeline is *architecturally* broken.

Ablation opportunities (future work).

I did not have the compute budget for these, but they are the direct next steps:

- **Projector warm-up on paired text.** Pre-train the projector on Whisper-encoded → LLM-embedded pairs of text-only utterances before the audio-loss training. Should break mode collapse in a few hundred steps.
- **Longer training.** 5-10 epochs on the full 90 h train set (~10-20k optimizer steps).
- **Larger LLM.** LLaMA-3-8B in 4-bit (13 GB) would still fit in VRAM if I drop the pipeline split; the extra language modelling capacity should help with English technical vocabulary.
- **Beam search.** Currently greedy. Beam size 4 typically halves absolute WER for LLM-based ASR.

Error analysis

WER vs code-switch intensity. Utterances in the test set were bucketed by their code-switch rate (number of language-changes per adjacent-token pair):

- ***Low switch rate (< 5 %)****:* monolingual or near-monolingual utterances.
- ***Mid (5-25 %)****:* 1-2 English words in a Hindi sentence, typical for MUCS.
- ***High (> 25 %)****:* heavy back-and-forth code-switching.

Because SLAM-ASR is mode-collapsed, WER is uniform across all three buckets (~2.0). A converged model would show a clear positive correlation between switch rate and WER; ours does not because the output is independent of input.

Script confusion (test set). Because the model emits the same seed sentence regardless of input, the token-level confusion by script is heavily skewed:

- ***Reference Hindi → Hypothesis Hindi:*** high (both contain some Hindi tokens by chance)
- ***Reference English → Hypothesis Hindi:*** high (the seed has no English)
- ***Reference Hindi → Hypothesis English:*** near-zero
- ***Reference English → Hypothesis English:*** near-zero (English WER = 1.00 confirms)

Long-utterance behaviour. Utterances longer than 15 s were filtered out at inference (matching training). Full-length decoding (up to 25 s) would require re-training with `FAST_MAX_DURATION = 25` and a larger `n_audio_tokens`.

Limitations and future work

Compute. 1-2 GPU-hours is ~200× less than the reference paper. The dominant failure mode (projector mode collapse) is a direct consequence.

Data. 3 hours of the 90-hour train set which is 3 % of what's available. Full-set training in a single Kaggle session is infeasible; requires either multiple sessions with resume, or migration to Colab Pro / a paid cloud GPU.

Text-only projector warm-up. Not implemented. This is the standard first-stage in the SLAM-LLM recipe and is likely the single most impactful next step.

Beam search. Currently greedy. Adding num_beams=4 with a modest length penalty should reduce absolute WER by ~20-30 %.

Domain. MUCS is technical-lecture speech from a specific programme (Spoken Tutorial Project). Generalisation to conversational Hindi-English (e.g., WhatsApp voice notes) is unknown.

Ethics. Speaker identity is potentially recoverable from Whisper encoder features, though I freeze and never fine-tune on speaker labels. Downstream users should not deploy the model in adversarial contexts (identity verification, surveillance).

Reproducibility

GitHub: https://github.com/kraviteja95/slam-asr-code-switched-speech-recognition

Kaggle notebooks (Private):

- ***Training + evaluation (all-in-one):*** notebooks/slam_asr_all_in_one_kaggle.ipynb
- ***Inference demo:*** notebooks/slam_asr_inferencing_demo_transcribe.ipynb

Dataset: kraviteja95/mucs-2021-hindi-english-code-switched-speech (Kaggle, private)

Checkpoint: release/slam_asr_outputs/checkpoints/slam_asr/final/ (91 MB - LoRA safetensors + projector.pt; frozen backbones re-download from HuggingFace at openai/whisper-base and Qwen/Qwen2.5-1.5B-Instruct).

Predictions: release/slam_asr_outputs/predictions/{test,blindtest}.jsonl (300 utts each).

Metrics: release/slam_asr_outputs/results/summary.json.

Random seed: 42 throughout (Python, NumPy, PyTorch).

Hardware used: Kaggle T4 x2, ~90 min wall-clock end-to-end.

Conclusion

I designed, implemented and evaluated an end-to-end SLAM-ASR system for Hindi-English code-switched speech on the MUCS 2021 corpus. The system is **complete, publicly reproducible, and covers every topic of the 5-day speech course**, from psychoacoustic feature choice through classical GMM baselines, char-level CTC, Whisper zero-shot/fine-tune, and the SLAM-LLM tripartite architecture. At our compute budget of one free Kaggle session, no model in the comparison achieves competitive WER; I characterised the specific failure mode of each and identified **projector mode collapse due to severe under-training** as the SLAM-ASR bottleneck. The pipeline is architecturally sound and ready for scale-up experiments; the trained checkpoint, all predictions, and both notebooks are released for future work.

References

- **Radford, A., et al.** *Robust Speech Recognition via Large-Scale Weak Supervision.* OpenAI Whisper technical report, 2022.
- **Ma, Z., et al.** *An Embarrassingly Simple Approach for LLM with Strong ASR Capacity.* arXiv:2402.08846, 2024.
- **Hu, E., et al.** *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR 2022.
- **Dettmers, T., et al.** *QLoRA: Efficient Finetuning of Quantized LLMs.* NeurIPS 2023.
- **Graves, A., et al.** *Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks.* ICML 2006.
- **Diwan, A., et al.** *Multilingual and Code-Switching ASR Challenges for Low-Resource Indian Languages.* INTERSPEECH 2021.
- **Traunmüller, H.** *Analytical expressions for the tonotopic sensory scale.* JASA 1990.
- **Terhardt, E.** *Calculating virtual pitch.* Hearing Research, 1979.

Appendix

Appendix A - Full architecture code layout

The trained pipeline is a Python package with the following structure (present inside `release/slam_asr_outputs/src/` - bootstrapped into every Kaggle session from the notebook's embedded JSON blob):

```
src/
├── data/
│   ├── manifest_utils.py       ← JSONL I/O, path rewriter, dataset statistics
│   ├── text_normalization.py   ← CodeSwitchTextNormalizer (WER-critical)
│   └── dataset.py              ← MUCSDataset + WhisperCollator + SlamASRCollator
├── features/
│   ├── audio_features.py       ← STFT, log-Mel, MFCC, chroma, centroid, ZCR
│   └── psychoacoustic.py       ← Mel/Bark, ATH, masking curves
├── models/
│   ├── projector.py            ← 1D-CNN + MLP alignment projector
│   ├── slam_asr.py             ← SlamAsrModel + SlamAsrConfig
│   └── baselines.py            ← GMM keyword spotter + Bi-LSTM CTC
├── training/
│   └── train_slam.py           ← HF-Trainer wrapper with per-group LRs
├── inference/
│   └── decode.py               ← decode_slam_asr + decode_manifest + CLI
├── evaluation/
│   ├── metrics.py              ← compute_wer_cer + per_language_wer + oov_rate
│   └── code_switch_analysis.py ← code_switch_stats + confusion_by_script
└── demo/
    └── gradio_app.py           ← optional Gradio 2-tab demo
```

All ~2 000 lines of Python are extensively docstring-documented and unit-testable.

Appendix B - Key hyperparameters

SLAM-ASR configuration (also in release/slam_asr_outputs/checkpoints/slam_asr/final/slam_config.json):

```
{
  "encoder_name":              "openai/whisper-base",
  "decoder_name":              "Qwen/Qwen2.5-1.5B-Instruct",
  "projector_hidden_dim":      2048,
  "projector_downsample_factor": 5,
  "projector_n_conv_layers":   1,
  "projector_dropout":         0.0,
  "load_in_4bit":              true,
  "torch_dtype":               "bfloat16",
  "use_lora":                  true,
  "lora_r":                    16,
  "lora_alpha":                32,
  "lora_dropout":              0.05,
  "lora_target_modules":       ["q_proj","k_proj","v_proj","o_proj",
                               "up_proj","down_proj","gate_proj"],
  "freeze_encoder":            true,
  "freeze_llm_backbone":       true
}
```

Training run settings:

- SLAM_TRAIN_HOURS = 3.0 (1 881 utterances)
- SLAM_BATCH_SIZE = 2, GRAD_ACCUM_STEPS = 4 (effective batch 8)
- SLAM_EPOCHS = 1 (~236 optimizer steps)
- n_audio_tokens = 150 (max audio-token span in the LLM prompt)
- max_duration_s = 15 (audio-duration filter for training)
- LR: projector 1e-3, LoRA 1e-4; cosine schedule with 3 % warmup
- bf16, gradient checkpointing (use_reentrant=False)
- device_map="auto" for the LLM (pipeline split across the two T4s)

Evaluation settings:

- 300 utterances from test, 300 from blindtest
- max_new_tokens = 100, num_beams = 1 (greedy)
- Single GPU (cuda:0) for inference
- n_audio_tokens = 150 (matches training)

Appendix C - Kaggle notebooks

Both notebooks live under notebooks/ in the GitHub repo and are 100 % self-contained (they bootstrap src/ from an embedded JSON blob; no git clone and no extra source-code Kaggle Dataset required).

- **Training + evaluation:** `notebooks/slam_asr_all_in_one_kaggle.ipynb`
- **Inference demo:** `notebooks/slam_asr_inferencing_demo_transcribe.ipynb`

Both notebooks are shipped with **cell outputs baked in**, so reviewers can inspect exactly what happened without re-executing them.